МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №4

з курсу: „Паралельні обчислення”
тема: “ Дослідження міжпроцесної взаємодії ”

виконав студент 3го курсу
групи ДА-11
Сидько Нікіта Сергійович

КИЇВ – 2024

Мета роботи:Детально розглянути поняття процесу, ресурсів процесу, а також підходи до міжпроцесової взаємодії, як в межах одного ПК, так і на різних ПК. Отримати практичні навички організації міжпроцесової взаємодії на прикладі написання клієнт-серверного застосунку.

Завдання

1. Розробити клієнт-серверний застосунок для вирішення завдання з лабораторний робот номер 1, передавши масив даних з клієнта на сервер, а потім – отримавши результат назад на сторону клієнта. Для виконання основного завдання дозволено використовувати лише платформні (WinSock, POSIX) та вбудовані засоби роботи з сокетами.

2. Розробити протокол прикладного рівня для взаємодії клієнта з сервером. Для цього врахувати декілька кроків в процесі взаємодії. Надсилання даних та конфігурації обчислень, надсилання команди (та отримання відповіді на команду) для початку обчислень та команди для запиту статусу і результату обчислень.

3. Додати до розробленого сервер підтримку підключення декількох клієнтів одночасно. Додатковий бал можна отримати у випадку наявності двох клієнтів, один з яких буде розроблений на мові, відмінній від мови серверу та першого клієнту (другий клієнт дозволено створювати з використанням скриптових мов).

4. Застосунок повинен коректно оброблювати виняткові ситуації як на стороні клієнту, так і на стороні серверу і адекватно реагувати на них. Без завершення своєї роботи. Обов’язковим є коректна обробка порядку кодування байтів у повідомленні.

5. В протоколі роботи необхідно навести опис розробленого застосунку. До цього опису повинні входити обґрунтування вибору протоколу передачі даних, а також архітектурний опис клієнта.

6. Занести до протоколу роботи опис розробленого протоколу прикладного рівня у вигляді таблиці, що включає: перелік всіх команд, аргументи команд та їх опис, список можливих відповідей на команду.

7. Занести до протоколу роботи UML діаграму викликів взаємодії серверу та клієнту, починаючи від запуску клієнту, до завершення роботи.

8. Надати висновок, що повинен містити аналіз та опис проблем з котрими зіштовхнувся студент, або з якими може зіштовхнутися розробник при організації міжпроцесової взаємодії.

Сервер – комп'ютер/програма, яка надає ресурси, дані, послуги іншим програмам або пристроям, що називаються клієнтами. Сервери можуть виконувати різноманітні функції, такі як зберігання файлів, обробка запитів на веб-сторінки або управління електронною поштою.

Сокет – це механізм в програмуванні для обміну даними між процесами через мережу. Вони використовуються для створення з'єднань між клієнтами і серверами та передачі даних між ними.

TCP (Transmission Control Protocol) і UDP (User Datagram Protocol) - це два основних протоколи транспортного рівня в мережевій моделі OSI (Open Systems Interconnection).

TCP

- TCP є з'єднанням та орієнтованим на потік протоколом. Він забезпечує надійну, послідовну та орієнтовану на потік передачу даних між двома пристроями.
- TCP використовує механізми як встановлення з'єднання, контролю потоку, підтвердження отриманих пакетів та перевірки цілісності даних.

UDP є простим протоколом без з'єднання та без забезпечення надійної доставки даних.

- Він передає незалежні пакети даних без будь-якого контролю потоку, підтвердження доставки або відновлення порядку.
- UDP швидше за TCP та менше витрачає ресурси мережі, але дозволяє втрату даних або їх дублювання.

Тобто TCP більш надійний протокол, але при цьому менш ефективним у використанні ресурсів мережі порівняно з UDP.

Для данної лабораторної роботи було написано код клієнта та код серверу, які з’єднуются за допомогою TCP протоколу адже він забезпечує надійну, послідовну та орієнтовану на потік передачу.

Код серверу:

Ініціалізація Winsock:
Сервер використовує WSAStartup() для ініціалізації використання Winsock DLL.

Створення сокета:
Виклик socket() створює сокет, який слухає вхідні TCP з'єднання. Сокет налаштовується на роботу з IPv4 (AF_INET) та потоковим типом передачі (SOCK_STREAM).

Прив'язка сокета:
Сервер прив'язує сокет до певного порту на локальному комп'ютері за допомогою функції bind(). Це вказує на те, що сервер буде слухати вхідні з'єднання на цьому порту.

Слухання:
Функція listen() налаштовує сокет на слухання вхідних з'єднань, визначаючи максимальну кількість клієнтських з'єднань, які можуть бути оброблені в черзі.

Прийняття з'єднань:
Сервер чекає на вхідне з'єднання з клієнтом за допомогою accept(), яка блокується, поки клієнт не з'єднається. Коли клієнт з'єднується, accept() повертає новий сокет, специфічний для цього з'єднання, через який відбувається подальше спілкування.
Для передачі даних від сервера до клієнта та з клієнта до сервера використаємо стандартну бібліотеку Winsock для програм на платформі Windows.

Почнемо з коду сервера, де було реалізовано дві функції

```
void Compute(std::vector<std::vector<int>>& matrix, int start, int end) {
    for (int i = start; i < end; ++i) {
        for (int j = 0; j < matrix.size(); ++j) {
            std::swap( &: matrix[i][j],  &: matrix[matrix.size() - 1 - i][j]);
        }
    }
}
```

Це функція обробки масиву (відзеркалення) по частинам (від start до end) де в подальшому матриця буде розбиватися на частини і оброблятися потоками, де в кожному потоці буде наявна ця функція.

А також функція обробки клієнта

```
void handle_client(SOCKET clientSocket, int clientNumber) {
    int size;
    recv( s: clientSocket, buf: (char*)&size, len: sizeof(size), flags: 0);

    std::vector<std::vector<int>> matrix ( n: size, value: std::vector<int>( n: size));
    for (auto& row : vector<int> &  : matrix) {
        recv( s: clientSocket, buf: (char*)row.data(), len: sizeof(int) * row.size(), flags: 0);
    }
```

На початку приймає розмір матриці, та ініціалізує матрицю клієнта

```
auto start_time : time_point<...>  = std::chrono::high_resolution_clock::now();
std::vector<std::thread> threads;
int segment = size / NUM_THREADS;
for (int i = 0; i < NUM_THREADS; ++i) {
    int start = i * segment;
    int end = (i + 1) * segment;
    if (i == NUM_THREADS - 1) end = size;
    threads.emplace_back( &: Compute, std::ref( &: matrix), start, end);
}
for (auto& thread : thread &  : threads)
{
    thread.join();
}

auto end_time : time_point<...>  = std::chrono::high_resolution_clock::now();
std::chrono::duration<double> elapsed = end_time - start_time;
```

Далі створюється вектор потоків, що будуть обробляти матрицю. І кожен потік обробляє свою частину матриці. Також виконується замір часу обробки.

```
    for (const auto& row : vector<int> const &  : matrix) {
        send( s: clientSocket, buf: (char*)row.data(), len: sizeof(int) * row.size(), flags: 0);
    }

    std::lock_guard<std::mutex> lock( &: cout_mutex);
    std::cout << "Client " << clientNumber << " (Socket: " << clientSocket << "): Matrix proc

    closesocket( s: clientSocket);
}
```

Далі сервер надсилає клієнту оброблену матрицю, та відбувається закриття сокету.

Далі перейдемо до main ()

```
int main() {
    WSADATA wsaData;
    WSAStartup( wVersionRequested: MAKEWORD(2, 2), lpWSAData: &wsaData);
    SOCKET ListenSocket = socket( af: AF_INET, type: SOCK_STREAM, protocol: IPPROTO_TCP);
    sockaddr_in service;
    service.sin_family = AF_INET;
    service.sin_addr.s_addr = INADDR_ANY;
    service.sin_port = htons( hostshort: 27015);
    bind( s: ListenSocket, name: (SOCKADDR *)&service, namelen: sizeof(service));
    listen( s: ListenSocket, backlog: SOMAXCONN);
```

Де в нас реалізовано наступне :
WSADATA wsaData - зберігає інформацію про реалізацію Windows Sockets.
WSAStartup(MAKEWORD(2, 2), &wsaData) - ініціалізує використання Winsock DLL версії 2.2. Це необхідно для початку роботи з сокетами у Windows.
SOCKET ListenSocket - Створює сокет для прослуховування вхідних з'єднань. socket(AF_INET, SOCK_STREAM, IPPROTO_TCP) вказує, що сокет буде використовувати IPv4 адреси (AF_INET), буде потоковим сокетом (SOCK_STREAM), і використовуватиме TCP протокол (IPPROTO_TCP).

sockaddr_in service - структура, яка зберігає адресу для сокета.
service.sin_family = AF_INET; - Вказує, що адреса використовує IPv4.
service.sin_addr.s_addr = INADDR_ANY - сокет буде зв'язаний з усіма доступними інтерфейсами.
service.sin_port = htons(27015) - Встановлює порт 27015 для прослуховування, конвертуючи число з порядку байт хоста у порядок байт мережі.

bind() - Прив'язує сокет ListenSocket до адреси, вказаної в service.

listen() - Розпочинає прослуховування на сокеті ListenSocket, дозволяючи максимальну кількість з'єднань, яка може бути оброблена одночасно

```
while (true) {
    SOCKET ClientSocket = accept( s: ListenSocket, addr: NULL, addrlen: NULL);
    if (ClientSocket == INVALID_SOCKET) continue;
    std::thread clientThread( &: handle_client, ClientSocket, ++clientNumber);
    clientThread.detach();
}
```

Також там реалізовано нескінченний цикл (щоб сервер постійно очікував на клієнта) де сервер чекає і приймає вхідне з'єднання, створюючи новий сокет ClientSocket для кожного з'єднання. Далі створюється поток для клієнта, та прописаний detach( ) для того щоб один клієнський потік не очікував на інші.

Код клієнта

```cpp
#include <iostream>
#include <winsock2.h>
#include <ws2tcpip.h>
#include <vector>
#include <cstdlib>
#include <thread>
#include <mutex>

#pragma comment(lib, "ws2_32.lib")

std::mutex cout_mutex;

std::vector<std::vector<int>> generate_matrix(int size) {
    std::vector<std::vector<int>> matrix( n: size, value: std::vector<int>( n: size));
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            matrix[i][j] = rand() % 10;
        }
    }
    return matrix;
}
```

Дана функція генерує матрицю заповнюючи її числами.

```cpp
void send_matrix(const SOCKET& socket, const std::vector<std::vector<int>>& matrix) {
    int size = matrix.size();
    send( s: socket, buf: (char*)&size, len: sizeof(size), flags: 0);
    for (const auto& row : vector<int> const & : matrix) {
        send( s: socket, buf: (char*)row.data(), len: sizeof(int) * row.size(), flags: 0);
    }
}
```

Далі йде функція відправлення матриці. Ця функція відправляє матрицю через вказаний сокет. Вона спочатку відправляє розмір матриці, а потім відправляє дані кожного рядка матриці.

```cpp
std::vector<std::vector<int>> receive_matrix(const SOCKET& socket, int size) {
    std::vector<std::vector<int>> matrix( n: size, value: std::vector<int>( n: size));
    for (auto& row : vector<int> & : matrix) {
        recv( s: socket, buf: (char*)row.data(), len: sizeof(int) * row.size(), flags: 0);
    }
    return matrix;
}
```

Функція отримання матриці від сервера. Ця функція приймає матрицю зазначеного розміру через сокет. Вона створює матрицю відповідного розміру та читає дані кожного рядка з сокета.

```
void client_process(int client_id) {
    int size = 1000 + client_id * 1000;
    WSADATA wsaData;
    WSAStartup( wVersionRequested: MAKEWORD(2, 2), lpWSAData: &wsaData);
    SOCKET ConnectSocket = socket( af: AF_INET, type: SOCK_STREAM, protocol: IPPROTO_TCP);
    sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_port = htons( hostshort: 27015);
    inet_pton( Family: AF_INET, pStringBuf: "192.168.0.106", pAddr: &addr.sin_addr);
    if (connect( s: ConnectSocket, name: (sockaddr*)&addr, namelen: sizeof(addr)) == SOCKET_ERROR) {
        std::lock_guard<std::mutex> lock( &: cout_mutex);
        std::cerr << "Client " << client_id << ": Server connection error." << std::endl;
        closesocket( s: ConnectSocket);
        WSACleanup();
        return;
    }
    auto matrix : vector<vector<int>> = generate_matrix(size);
    {
        std::lock_guard<std::mutex> lock( &: cout_mutex);
        std::cout << "Client " << client_id << ": Matrix generated and sent. Size: " << size <<" x "<< size << std::endl;
    }
    send_matrix( socket: ConnectSocket, matrix);
    auto received_matrix : vector<vector<int>> = receive_matrix( socket: ConnectSocket, size);
    {
        std::lock_guard<std::mutex> lock( &: cout_mutex);
        std::cout << "Client " << client_id << ": Processed matrix received." << std::endl;
    }
    closesocket( s: ConnectSocket);
    WSACleanup();
}
```

Далі йде функція роботи клієнта.

Функція приймає ідентифікатор клієнта як параметр, що використовується для налаштування унікальних параметрів, таких як розмір матриці. Функція створює новий сокет для TCP з'єднання, використовуючи IPv4, Налаштовує структуру адреси для підключення до сервера. Використовує IP адресу сервера та порт
Також там додана умова

```
if (connect( s: ConnectSocket, name: (sockaddr*)&addr, namelen: sizeof(addr)) == SOCKET_ERROR) {
    std::lock_guard<std::mutex> lock( &: cout_mutex);
    std::cerr << "Client " << client_id << ": Server connection error." << std::endl;
    closesocket( s: ConnectSocket);
    WSACleanup();
    return;
}
```

Якщо підключення не вдається, виводиться повідомлення про помилку, сокет закривається, виконується очищення Winsock і функція завершується.

Далі йде генерація матриці та відправляє матрицю на сервер через відкрите з'єднання. Після чого отримує оброблену матрицю від сервера та виводить повідомлення про її отримання.

Розробити клієнт-серверний застосунок

Клієнт генерує матрицю, відсилає її на сервер, сервер обробляє і відсилає назад
auto matrix = generate_matrix(size);
send_matrix(ConnectSocket, matrix);
auto received_matrix = receive_matrix(ConnectSocket, size);

Підтримка множинних клієнтів

```
while (true) {
    SOCKET ClientSocket = accept( s: ListenSocket,  addr: NULL,  addrlen: NULL);
    if (ClientSocket == INVALID_SOCKET){
        continue;
    }
    std::thread clientThread( &: handle_client, ClientSocket, ++clientNumber);
    clientThread.detach();
}
```

Обробка виняткових ситуацій

```
if (connect( s: ConnectSocket,  name: (sockaddr*)&addr,  namelen: sizeof(addr)) == SOCKET_ERROR) {
    std::lock_guard<std::mutex> lock( &: cout_mutex);
    std::cerr << "Client " << client_id << ": Server connection error." << std::endl;
    closesocket( s: ConnectSocket);
    WSACleanup();
    return;
}
```

Результат роботи коду:
Сервер

```
Server is waiting for clients...
Client 2 (Socket: 208): Matrix processed and sent back. Processing time: 0.0107991 seconds.
Client 3 (Socket: 216): Matrix processed and sent back. Processing time: 0.0257134 seconds.
Client 1 (Socket: 200): Matrix processed and sent back. Processing time: 0.0470686 seconds.
```

Клієнт

```
Enter number of clients: 3
Client 1: Matrix generated and sent. Size: 2000 x 2000
Client 1: Processed matrix received.
Client 2: Matrix generated and sent. Size: 3000 x 3000
Client 2: Processed matrix received.
Client 3: Matrix generated and sent. Size: 4000 x 4000
Client 3: Processed matrix received.

Process finished with exit code 0
```

Висновок

У ході виконання лабораторної роботи 4, було написано клієнт-серверний застосунок на мові C++ , були отримані практичні навички організації міжпроцесової взаємодії на прикладі написання клієнт-серверного застосунку. Для надання з’єднання був обраний TCP протокол через його надійність. У програмі було реалізовано можливість підлючення декількох клієнтів, для яких виділялися окремі потоки, а також реалізовано підрахунок часу за який оброблялася матриця. Програма була протестована на корректність виконання

Додаток

Код серверу

```
#include <iostream>
#include <winsock2.h>
#include <vector>
#include <thread>
#include <chrono>
#include <mutex>

#pragma comment(lib, "ws2_32.lib")

const int NUM_THREADS = 8;

std::mutex cout_mutex;

void Compute(std::vector<std::vector<int>>& matrix, int start,
int end) {
    for (int i = start; i < end; ++i) {
        for (int j = 0; j < matrix.size(); ++j) {
            std::swap(matrix[i][j], matrix[matrix.size() - 1 -
i][j]);
        }
    }
}

void handle_client(SOCKET clientSocket, int clientNumber) {
    int size;
    recv(clientSocket, (char*)&size, sizeof(size), 0); //
получаю размер матрици

    std::vector<std::vector<int>> matrix
(size,std::vector<int>(size)); // задаю размер вектора
    for (auto& row : matrix) {  // заполнение матрицы числами
клиента
        recv(clientSocket, (char*)row.data(), sizeof(int) *
row.size(), 0);
    }

    auto start_time = std::chrono::high_resolution_clock::now();
    std::vector<std::thread> threads; // вектор потоков
    int segment = size / NUM_THREADS;
```

```
    for (int i = 0; i < NUM_THREADS; ++i) {
        int start = i * segment;
        int end = (i + 1) * segment;
        if (i == NUM_THREADS - 1) end = size;
        threads.emplace_back(Compute, std::ref(matrix), start,
end);
    }
    for (auto& thread : threads)
    {
        thread.join();
    }

    auto end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end_time -
start_time;

    for (const auto& row : matrix) {
        send(clientSocket, (char*)row.data(), sizeof(int) *
row.size(), 0);
    }

    std::lock_guard<std::mutex> lock(cout_mutex);
    std::cout << "Client " << clientNumber << " (Socket: " <<
clientSocket << "): Matrix processed and sent back. Processing
time: " << elapsed.count() << " seconds." << std::endl;

    closesocket(clientSocket);
}

int main() {
    WSADATA wsaData;
    WSAStartup(MAKEWORD(2, 2), &wsaData);
    SOCKET ListenSocket = socket(AF_INET, SOCK_STREAM,
IPPROTO_TCP);
    sockaddr_in service;
    service.sin_family = AF_INET;
    service.sin_addr.s_addr = INADDR_ANY;
    service.sin_port = htons(27015);
    bind(ListenSocket, (SOCKADDR *)&service, sizeof(service));
    listen(ListenSocket, SOMAXCONN);
    std::cout << "Server is waiting for clients..." <<
std::endl;
    int clientNumber = 0;

    while (true) {
        SOCKET ClientSocket = accept(ListenSocket, NULL, NULL);
        if (ClientSocket == INVALID_SOCKET){
            continue;
        }
        std::thread clientThread(handle_client, ClientSocket, +
+clientNumber); // поток для клиента
        clientThread.detach();
    }
```

```
    closesocket(ListenSocket);
    WSACleanup();
    return 0;
}
```

Код клієнта

```
#include <iostream>
#include <winsock2.h>
#include <ws2tcpip.h>
#include <vector>
#include <cstdlib>
#include <thread>
#include <mutex>

#pragma comment(lib, "ws2_32.lib")

std::mutex cout_mutex;

std::vector<std::vector<int>> generate_matrix(int size) {
    std::vector<std::vector<int>> matrix(size,
std::vector<int>(size));
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            matrix[i][j] = rand() % 10;
        }
    }
    return matrix;
}

void send_matrix(const SOCKET& socket, const
std::vector<std::vector<int>>& matrix) {
    int size = matrix.size();
    send(socket, (char*)&size, sizeof(size), 0);
    for (const auto& row : matrix) {
        send(socket, (char*)row.data(), sizeof(int) *
row.size(), 0);
    }
}

std::vector<std::vector<int>> receive_matrix(const SOCKET&
socket, int size) {
    std::vector<std::vector<int>> matrix(size,
std::vector<int>(size));
    for (auto& row : matrix) {
        recv(socket, (char*)row.data(), sizeof(int) *
row.size(), 0);
    }
    return matrix;
}

void client_process(int client_id) {
    int size = 1000 + client_id * 1000;
    WSADATA wsaData;
```

```
    WSAStartup(MAKEWORD(2, 2), &wsaData);
    SOCKET ConnectSocket = socket(AF_INET, SOCK_STREAM,
IPPROTO_TCP);
    sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_port = htons(27015);
    inet_pton(AF_INET, "192.168.0.106", &addr.sin_addr);
    if (connect(ConnectSocket, (sockaddr*)&addr, sizeof(addr))
== SOCKET_ERROR) {
        std::lock_guard<std::mutex> lock(cout_mutex);
        std::cerr << "Client " << client_id << ": Server
connection error." << std::endl;
        closesocket(ConnectSocket);
        WSACleanup();
        return;
    }
    auto matrix = generate_matrix(size);
    {
        std::lock_guard<std::mutex> lock(cout_mutex);
        std::cout << "Client " << client_id << ": Matrix
generated and sent. Size: " << size <<" x "<< size << std::endl;
    }
    send_matrix(ConnectSocket, matrix);
    auto received_matrix = receive_matrix(ConnectSocket, size);
    {
        std::lock_guard<std::mutex> lock(cout_mutex);
        std::cout << "Client " << client_id << ": Processed
matrix received." << std::endl;
    }
    closesocket(ConnectSocket);
    WSACleanup();
}

int main() {
    int client_count;
    std::cout << "Enter number of clients: ";
    std::cin >> client_count;
    std::vector<std::thread> clients;
    for (int i = 0; i < client_count; i++) {
        clients.emplace_back(client_process, i + 1);
    }
    for (auto& client : clients) {
        client.join();
    }
    return 0;
}
```